

Full-Stack Social Media Platform

Identity, discovery, posts, reels, stories, notifications, and persistent real-time messaging

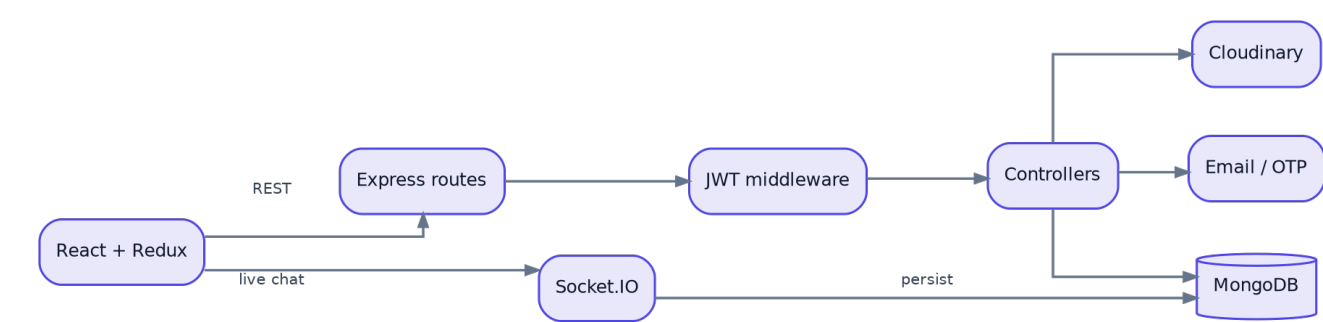
Project context: ACM IIT Roorkee full-stack project	Status: Implemented prototype
---	-------------------------------

A MERN social platform with email OTP signup, JWT login, profiles, search, follows, image posts, video reels, 24-hour stories, likes, comments, notifications, Cloudinary media storage, and direct messages that combine MongoDB persistence with Socket.IO delivery.

Technology stack

React	Redux Toolkit	Tailwind CSS	Express
MongoDB / Mongoose	Cloudinary	Socket.IO	JWT / Nodemailer

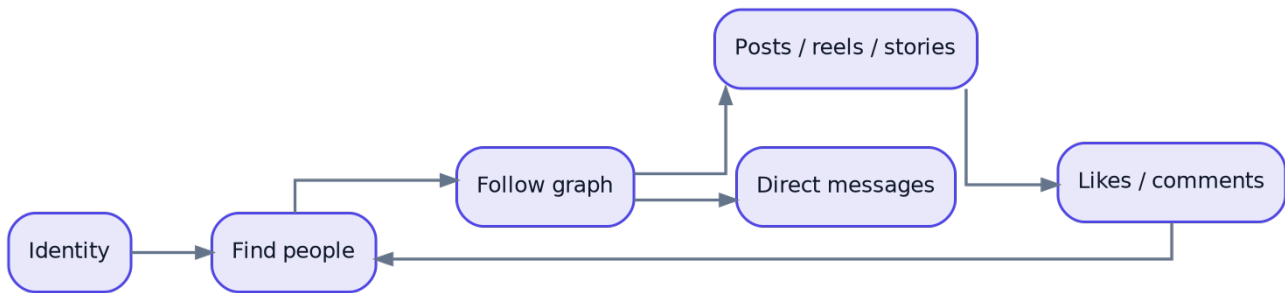
System at a glance



Identity and profiles OTP signup, login, profile editing, search, and display-picture upload.	Social graph Follow/unfollow flows, suggestions, profiles, and notifications.
Rich content Image posts, short-video reels, and expiring stories.	Messaging Stored conversations plus live Socket.IO delivery to active participants.

System design and component ownership

React screens and Redux state communicate with an Express API protected by JWT middleware on selected routes. Controllers use Mongoose models, Cloudinary, and email services. A Socket.IO layer handles chat-room events while MongoDB preserves message history.



Component	Responsibility
React + Redux	Route-level experiences, auth state, post state, and API calls.
Express routes	Maps URLs through middleware to feature controllers.
JWT middleware	Verifies identity for protected HTTP operations.
Mongoose models	Stores users, profiles, follows, content, notifications, and chats.
Cloudinary + mail	Hosts media and delivers OTP or account email.
Socket.IO	Joins chat rooms and broadcasts live messages.

Key design decisions

- Use Cloudinary URLs instead of storing media binaries in MongoDB.
- Model posts, reels, and stories separately because their lifecycles differ.
- Combine persistent chat records with live socket delivery.
- Keep the project positioned as a controlled-community prototype pending moderation and security work.

Core execution flow

Content creation uploads media first, stores a content record and owner relationship, and returns the created item to the browser. Chat derives a conversation, persists each message, and emits it to connected room members.



Implementation notes

- Stories are modeled as temporary content with a 24-hour lifecycle.
- React protected routes cover home, posts, reels, profiles, search, notifications, and messages.
- Cloudinary stores images and videos while MongoDB stores metadata and relationships.
- Prefix search supports member discovery.
- The README includes a one-command development flow for the UI and API.

Post creation sequence

```
Member -> React: caption + image + JWT
React -> Express: create-post request
Express -> Cloudinary: upload media
Cloudinary -> Express: hosted URL
Express -> MongoDB: save post and owner
MongoDB -> Express: created record
Express -> React: normalized success response
```

The sequence follows the documented repository interaction rather than reproducing controller code.

Engineering review and production path

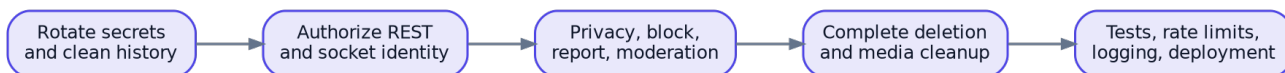
Engineering strengths

- Broad end-to-end product scope with multiple media formats.
- Clear React, Express, MongoDB, Cloudinary, and Socket.IO integration.
- Persistent plus live chat illustrates two consistency paths.
- Repository documentation includes a candid production-gap analysis.

Current limitations

- A populated environment file and uploaded artifacts are documented as tracked.
- Chat REST and socket identity checks need to be derived from verified JWTs.
- No moderation, block, report, privacy, or rate-limit workflow.
- Account deletion and media cleanup are incomplete; automated tests are absent.

Recommended next steps



Why this is relevant to AI training work

- Offers many nontrivial tasks involving identity, feeds, lifecycle, and relationship rules.
- Supports evaluation of generated code for authorization and data-consistency mistakes.
- Demonstrates distinction between functional completeness and production safety.
- Shows how to reason about multimedia, realtime, and persistent data together.

Repository:

<https://github.com/SidheshwarSarangal/social-media-website>

Scope and provenance:

Prepared from repository product, architecture, content/media, data/API, realtime-chat, setup, and current-boundary documentation.

